EKS AW

Desde AWS únicamente se puede crear el cluster, por lo cual es recomendado hacerlo todo en consola

1. Creación de cluster
2. Una vez creado el cluster desde AWS se debe conectar con el pc

```
aws eks update-kubeconfig --name eks-cesar --region us-east-1
```

2. Crear NodeGroup (son los nodos worker)
3.

creacion de vpc con cloud formation

usar:

```
https://s3.us-west-2.amazonaws.com/amazon-eks/cloudformation/
2020-10-29/amazon-eks-vpc-private-subnets.yaml
```

Aplicar Deployment:

```
kubectl apply -f deployment.yaml
```

para ver los servicios:

```
kubectl get svc
```

para eliminar el deployment

```
kubectl delete -f deployment.yaml
```

INSTALAR LOAD BALANCER EN EKS

1. creacion de politica

```
curl -o iam_policy.json https://raw.githubusercontent.com/kub
ernetes-sigs/aws-load-balancer-controller/main/docs/install/i
am_policy.json
```

2. Crear politica en mi cuenta:

```
aws iam create-policy --policy-name AWSLoadBalancerController
IAMPolicy --policy-document file://iam_policy.json
```

3. Crear el service Account en el EKS

```
eksctl create iamserviceaccount --cluster ce-cluster --namesp
ace kube-system --name aws-load-balancer-controller --attach-
policy-arn arn:aws:iam::240446479217:policy/AWSLoadBalancerCo
ntrollerIAMPolicy --approve
```

4. Instalar LoadBalancer en el servicio con Helm (pueda que solo necesites este paso)

```
helm repo add eks https://aws.github.io/eks-charts; helm repo
update
```

```
antes de instalar validar si ya existe en la lista de servici
os

helm list -n kube-system

si no instalar

helm install aws-load-balancer-controller eks/aws-load-balanc
er-controller -n kube-system --set clusterName=ce-cluster --s
et serviceAccount.create=false --set serviceAccount.name=aws-
load-balancer-controller --set region=us-east-1 --set vpcId=v
pc-07b30a0ecdef8731b --set subnets='{subnet-0b3308d22dcf9f84
9,subnet-0103c2d2a71a03047}'

si, si actualizar

eliminar primero antes de instalar
helm uninstall aws-load-balancer-controller -n kube-system
```

5. Validar si está instalado deben estar ready

```
kubectl get deployment -n kube-system aws-load-balancer-contr
oller
```

6. validar estado de los pods de ingress

```
kubectl get pods -n kube-system -l app.kubernetes.io/name=
aws-load-balancer-controller -o wide
```

AUTOESCALADO

1) Instalar metrics-server

Ejecuta (en tu terminal con `kubectl` configurado al clúster `ce-cluster`):

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-s
erver/releases/latest/download/components.yaml
```

Verifica que el APIService esté listo:

```
kubectl get apiservice v1beta1.metrics.k8s.io -o yaml | Selec
t-String "status"
```

2. agregar a los deployments los limites de los recursos

```yaml
# user-deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-deployment
  labels:
    app: user
spec:
  replicas: 2
  selector:
    matchLabels:
      app: user
  template:
    metadata:
      labels:
        app: user
    spec:
      containers:
        - name: user
          image: 240446479217.dkr.ecr.us-east-1.amazonaws.co
m/usuario:latest
```

```yaml
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "256Mi"
```

3. Crearle el horizontal scale

```yaml
# HPA para producto
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: producto-hpa
  namespace: default
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: producto-deployment
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

luego de aplicado validar que se esté ejecutando:

```
kubectl get hpa
```

Crear ServiceAccount IAM para Cluster Autoscaler

```
eksctl create iamserviceaccount --cluster ce-cluster --namesp
ace kube-system --name cluster-autoscaler --attach-policy-arn
arn:aws:iam::aws:policy/AutoScalingFullAccess --override-exis
ting-serviceaccounts --approve
```

5️⃣ Desplegar Cluster Autoscaler

```
# Descargar manifiesto oficial
Invoke-WebRequest -Uri "https://raw.githubusercontent.com/kub
ernetes/autoscaler/master/cluster-autoscaler/cloudprovider/aw
s/examples/cluster-autoscaler-autodiscover.yaml" -OutFile "cl
uster-autoscaler-autodiscover.yaml"
```

Busca esta sección en el `Deployment`:

```
- --node-group-auto-discovery=asg:tag=k8s.io/cluster-autoscal
er/enabled,k8s.io/cluster-autoscaler/<YOUR CLUSTER NAME>
```

Y reemplaza `<YOUR CLUSTER NAME>` por `ce-cluster`:

```
- --node-group-auto-discovery=asg:tag=k8s.io/cluster-autoscal
er/enabled,k8s.io/cluster-autoscaler/ce-cluster
```

✅ Esto indica al Cluster Autoscaler **qué AutoScalingGroups observar** para agregar o quitar nodos.

🔹 Resumen del paso final

1. Edita el YAML del Cluster Autoscaler.
2. Cambia `<YOUR CLUSTER NAME>` → `ce-cluster`.
3. Aplica el deployment:

```
kubectl apply -f cluster-autoscaler-autodiscover.yaml
```

1. Verifica que el pod se levante:

```
kubectl get pods -n kube-system | Select-String "cluster-auto
scaler"
```